UNIVERSIDADE FEDERAL DE OURO PRETO
CAMPUS MORRO DO CRUZEIRO – OURO PRETO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Disciplina: Programação Orientada a Objetos (BCC 221)
Professor: GUILLERMO CAMARA CHAVEZ

DAVID CRISTIAN DE PAULA RODRIGUES
MATHEUS CARDOSO DE ARAÚJO
PEDRO HENRIQUE BOSEJA CAROLINO BARBOSA
PEDRO SOUZA GOULARTE

DESENVOLVIMENTO DO JOGO TERMO EM JAVA COM INTERFACE GRÁFICA (SWING)

OURO PRETO
2026

Sumário

1 INTRODUÇÃO **2**

1.1 Objetivos do Trabalho 2

1.2 Escopo do Sistema Desenvolvido 2

2 DESCRIÇÃO DO SISTEMA **3**

2.1 Principais Funcionalidades 3

2.2 Entidades Manipuladas pelo Sistema 3

2.3 Interação do Usuário com a Interface Gráfica 4

3 DECISÕES DE PROJETO **4**

3.1 Modelagem Orientada a Objetos 4

3.2 Encapsulamento, Herança e Polimorfismo 5

3.3 Estruturas da Biblioteca Java Utilizadas 5

3.4 Persistência de Dados 6

4 ARQUITETURA DO PROJETO **7**

4.1 Descrição dos Pacotes 7

4.2 Funcionamento do Método Main 7

4.3 Relacionamento Entre Classes 8

4.4 Fluxo de Execução do Programa 8

5 DIAGRAMAS UML **10**

6 TESTES REALIZADOS **12**

6.1 Teste 1 – Vitória na Partida 13

6.2 Teste 2 – Acesso às Estatísticas 14

6.3 Teste 3 – Persistência Após Reinicialização 15

6.4 Teste 4 – Login com Senha Incorreta 16

6.5 Teste 5 – Palavra Fora do Dicionário 17

7 CONCLUSÃO **17**

1. INTRODUÇÃO

O presente trabalho aborda o desenvolvimento do jogo TERMO, inspirado no jogo Wordle, utilizando a linguagem Java e a biblioteca Swing para construção da interface gráfica. O problema proposto consiste em implementar um sistema capaz de selecionar aleatoriamente uma palavra secreta de cinco letras e permitir que o jogador tente descobri-la em até seis tentativas, recebendo feedback visual imediato sobre cada letra digitada.

O desafio do projeto não se limita apenas à implementação da mecânica do jogo, mas envolve também a aplicação prática dos principais conceitos de Programação Orientada a Objetos, como encapsulamento, composição, associação entre classes, herança, polimorfismo e manipulação de arquivos.

1.1 Objetivos do Trabalho

O objetivo geral deste trabalho é desenvolver uma aplicação completa, funcional e organizada do jogo TERMO, aplicando os conceitos estudados na disciplina de Programação Orientada a Objetos.

Como objetivos específicos, destacam-se:

- Modelar adequadamente as entidades do domínio do problema.
- Implementar uma interface gráfica interativa utilizando a biblioteca Swing.
- Aplicar o padrão arquitetural MVC (Model-View-Controller).
- Implementar persistência de dados utilizando serialização.
- Desenvolver um sistema de autenticação de usuários com armazenamento seguro de senhas por meio de hash SHA-256.

1.2 Escopo do Sistema Desenvolvido

O sistema desenvolvido contempla:

- Mecânica completa do jogo TERMO.
- Banco de palavras carregado a partir de arquivo externo.

- Controle de tentativas e validação de palavras.
- Feedback visual por meio de cores (verde, amarelo e cinza).
- Sistema de login e cadastro de usuários.
- Persistência de estatísticas individuais.
- Tela de estatísticas com histórico de desempenho.

2. DESCRIÇÃO DO SISTEMA

O sistema desenvolvido simula integralmente a mecânica do jogo TERMO, permitindo ao usuário tentar adivinhar uma palavra secreta de cinco letras em até seis tentativas.

2.1 Principais Funcionalidades

Entre as principais funcionalidades implementadas, destacam-se:

- Sorteio aleatório da palavra secreta a partir do banco de palavras.
- Validação da palavra digitada pelo jogador.
- Atualização visual da grade de tentativas.
- Atualização dinâmica do teclado virtual.
- Controle do número máximo de seis tentativas.
- Registro automático de vitória ou derrota.
- Sistema de autenticação com login e cadastro.
- Persistência das estatísticas do jogador.

2.2 Entidades Manipuladas pelo Sistema

O sistema manipula as seguintes entidades principais:

- **Letra**: Representa cada caractere individual da palavra, armazenando seu estado (correta, presente ou ausente).

- **Palavra**: Representa tanto a palavra secreta quanto as tentativas do jogador.
- **Partida**: Controla o estado atual do jogo, incluindo tentativas realizadas e verificação de vitória.
- **Jogador**: Representa o usuário autenticado.
- **Estatisticas**: Armazena dados como número de partidas jogadas, vitórias, derrotas e distribuição de tentativas.
- **BancoPalavras**: Responsável por carregar e fornecer palavras válidas.

2.3 Interação do Usuário com a Interface Gráfica

A interação ocorre por meio da classe `JanelaPrincipal`, onde o jogador pode:

- Digitar palavras utilizando teclado físico ou teclado virtual.
- Visualizar as cores indicativas após cada tentativa.
- Acessar o menu para visualizar estatísticas.
- Iniciar nova partida.
- Realizar login e cadastro.

O feedback visual utiliza cores padronizadas:

- Verde: letra correta na posição correta.
- Amarelo: letra existente em posição incorreta.
- Cinza: letra inexistente na palavra.

3. DECISÕES DE PROJETO

3.1 Modelagem Orientada a Objetos

A modelagem foi baseada na separação clara das responsabilidades. Optou-se por não utilizar diretamente objetos do tipo String como representação principal da lógica do jogo, mas sim encapsular o comportamento dentro das classes Palavra e Letra.

A classe Partida possui composição com Palavra, que por sua vez possui composição com Letra, garantindo que essas entidades não existam de forma independente.

Como exemplo da responsabilidade atribuída à classe `Palavra`, destaca-se o método de normalização, responsável por remover acentos e padronizar a entrada do usuário antes do processamento:

Listing 1: Metodo de normalizacao da palavra

```
public static String normalizarPalavra(String palavra) {
    if (palavra == null) return "";
    String resultado = palavra.toUpperCase().trim();
    return resultado.replace("Á","A").replace("Ã","A").replace("Â","A")
                    .replace("É","E").replace("Ê","E")
                    .replace("Í","I")
                    .replace("Ó","O").replace("Õ","O").replace("Ô","O")
                    .replace("Ú","U").replace("Ç","C");
}
```

Esse método garante que o sistema trate corretamente palavras com acentuação, mantendo conformidade com as regras do jogo.

3.2 Encapsulamento, Herança e Polimorfismo

O encapsulamento foi aplicado ao restringir o acesso direto aos atributos das classes, utilizando métodos getters e setters controlados.

O polimorfismo é observado na utilização da interface JogoListener, implementada pela camada View, permitindo que o ControladorJogo notifique eventos sem conhecer detalhes da interface gráfica.

A herança foi utilizada na criação de componentes personalizados da interface gráfica, que estendem classes como JButton e JPanel para customização visual.

3.3 Estruturas da Biblioteca Java Utilizadas

Foram utilizadas as seguintes estruturas:

- **HashMap<String, Jogador>** para armazenamento eficiente de usuários.

- **ArrayList<String>** para banco de palavras.
- **Serializable** para persistência de objetos.

Na interface Swing, foram utilizados:

- JFrame
- JPanel
- JButton
- JDialog
- JMenuBar

3.4 Persistência de Dados

A persistência foi implementada por meio da serialização de objetos Java. O sistema grava o mapa de usuários em arquivo binário (.dat).

As senhas não são armazenadas em texto plano, sendo convertidas para hash SHA-256 antes do armazenamento, aumentando a segurança do sistema.

A gravação dos dados dos usuários é realizada por meio da serialização:

Listing 2: Persistencia de usuarios

```
public void salvarUsuarios() {
    try (ObjectOutputStream out =
         new ObjectOutputStream(new FileOutputStream("usuarios.dat"))) {
        out.writeObject(mapaUsuarios);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Esse mecanismo garante que as estatísticas permaneçam armazenadas mesmo após o encerramento do sistema.

Para garantir segurança no armazenamento das senhas, foi utilizado o algoritmo SHA-256:

Listing 3: Geracao de hash SHA-256

```
private String gerarHashSenha(String senha) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(senha.getBytes());
        return Base64.getEncoder().encodeToString(hash);
    } catch (NoSuchAlgorithmException e) {
        return null;
    }
}
```

Dessa forma, mesmo que o arquivo de usuários seja acessado indevidamente, as senhas reais não podem ser recuperadas.

4. ARQUITETURA DO PROJETO

O sistema segue o padrão arquitetural MVC (Model-View-Controller).

4.1 Descrição dos Pacotes

- **model**: Contém as regras de negócio e entidades do sistema.
- **view**: Responsável pela interface gráfica.
- **controller**: Gerencia a comunicação entre Model e View.
- **util**: Contém classes auxiliares.

4.2 Funcionamento do Método Main

O método main realiza:

1. Leitura do argumento contendo o caminho do banco de palavras.
2. Inicialização do BancoPalavras.
3. Instanciação do ControladorJogo.
4. Inicialização da JanelaPrincipal.

5. Registro da View como observadora do Controller.

4.3 Relacionamento Entre Classes

O ControladorJogo atua como núcleo central, mantendo associação com Partida, BancoPalavras e GerenciadorUsuarios.

A View depende exclusivamente do Controller, não acessando diretamente a camada Model.

4.4 Fluxo de Execução do Programa

O fluxo simplificado do sistema pode ser descrito como:

Main → Inicializa BancoPalavras → ControladorJogo → JanelaPrincipal

Durante a partida:

Usuário → View → Controller → Model → Controller → View

A lógica central de processamento das tentativas ocorre no controlador, conforme exemplo simplificado abaixo:

Listing 4: Processamento de tentativa no controlador

```
public void processarTentativa(String entrada) {
    if (!bancoPalavras.isPalavraValida(entrada)) {
        view.mostrarErro("Palavra nao encontrada");
        return;
    }

    Palavra tentativa = new Palavra(entrada);
    partida.avaliarTentativa(tentativa);

    if (partida.isVitoria()) {
        view.mostrarVitoria();
    }
}
```

Esse método evidencia a separação de responsabilidades entre validação, regra de negócio e atualização da interface.

5. DIAGRAMAS UML

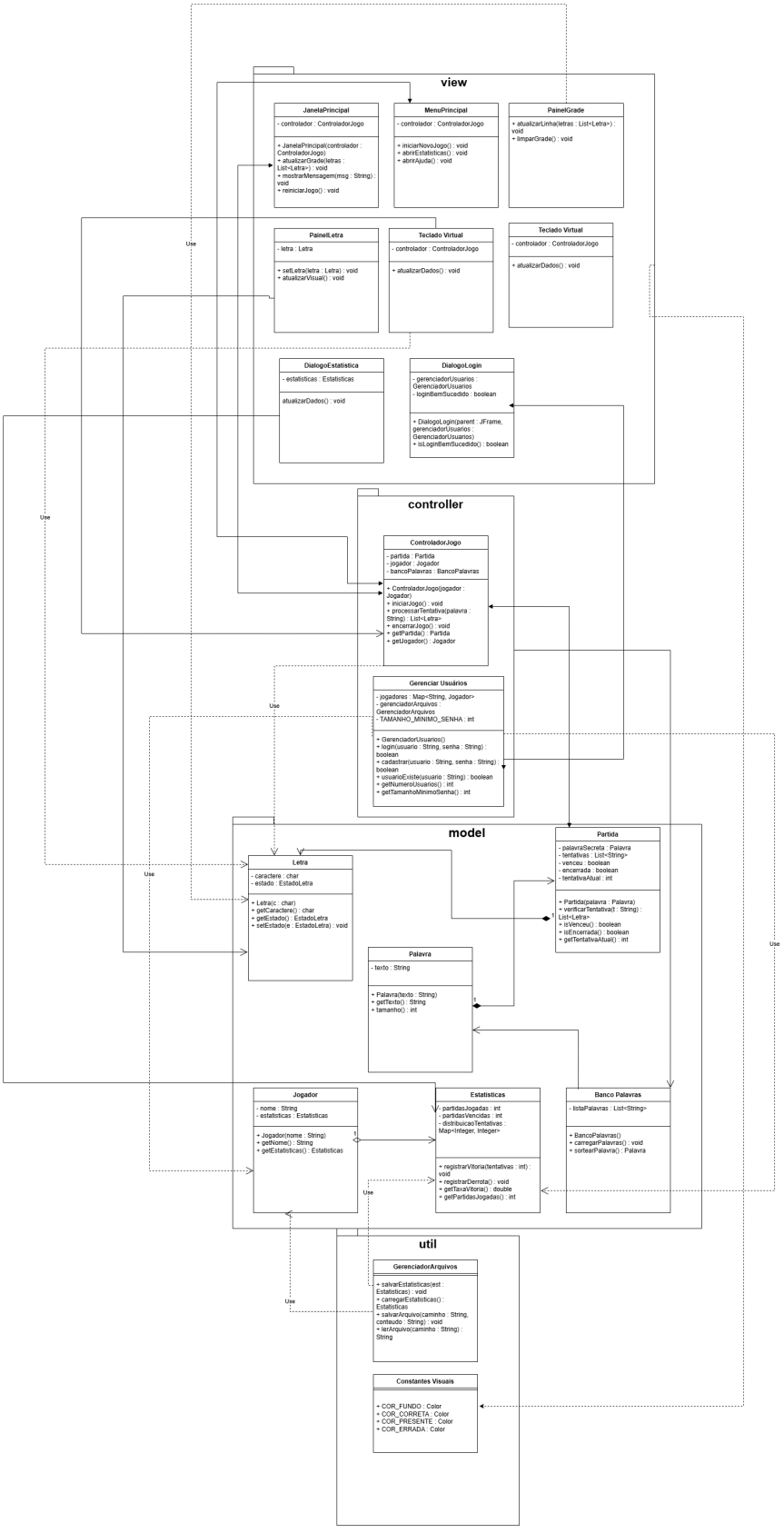


Figura 1: Diagrama de Classes do Sistema TERMO

Para melhor visualização do diagrama em alta resolução, recomenda-se acessar a versão completa disponível no link abaixo:

https://drive.google.com/file/d/1hJh1O8ZmqW8MqEtw2WSP692iCYzeeTFD/view?usp=sharing

O Diagrama de Classes apresentado evidencia claramente a organização estrutural do sistema segundo o padrão arquitetural MVC (Model-View-Controller), adotado como base para o desenvolvimento do projeto.

Na camada **Model**, observa-se a aplicação do princípio da composição entre as classes `Partida`, `Palavra` e `Letra`. A classe `Partida` encapsula a lógica central do jogo, sendo responsável pelo controle da palavra secreta, número de tentativas e verificação de vitória ou derrota. A classe `Palavra` representa uma estrutura composta por múltiplas instâncias de `Letra`, cada uma armazenando o caractere correspondente e seu estado (correta, presente em posição incorreta ou ausente).

A camada **Controller**, representada principalmente pela classe `ControladorJogo`, atua como intermediária entre Model e View. Essa classe centraliza o fluxo da aplicação, processando tentativas do jogador, validando palavras no banco de dados, atualizando estatísticas e notificando a interface gráfica sobre mudanças de estado.

Na camada **View**, as classes responsáveis pela interface gráfica dependem exclusivamente do controlador, não acessando diretamente as classes da camada Model. Essa separação reduz o acoplamento e melhora a organização estrutural do sistema, facilitando manutenção e expansão futura.

O diagrama também demonstra explicitamente:

- Relação de **composição** entre `Partida` e `Palavra`;
- Relação de **composição** entre `Palavra` e `Letra`;
- Relação de **associação** entre `ControladorJogo` e as entidades do Model;
- Implementação do padrão **Observer**, permitindo a comunicação desacoplada entre Controller e View.

Essa modelagem demonstra aplicação consistente dos princípios de encapsulamento, responsabilidade única, modularidade e separação de camadas, fundamentais na Programação Orientada a Objetos.

6. TESTES REALIZADOS

Foram realizados diversos testes manuais com o objetivo de validar não apenas a mecânica central do jogo, mas também a persistência de dados, o sistema de autenticação, o tratamento de erros e a consistência da interface gráfica.

Ao longo do processo de validação, foram executadas múltiplas partidas completas, com várias palavras do banco de dados testadas. Foram simulados diferentes cenários, incluindo vitórias em diferentes números de tentativas, derrotas após o limite máximo de seis tentativas, inserção de palavras inválidas, utilização de letras repetidas, reinicialização do sistema para verificação da persistência e tentativas de login com credenciais incorretas.

Além disso, foram realizados diversos ciclos de abertura e fechamento do sistema para assegurar que os dados fossem corretamente gravados e recuperados do arquivo binário. O comportamento do teclado virtual, atualização das cores e regras de distribuição de letras também foi cuidadosamente observado em diferentes situações.

A seguir, são apresentados alguns dos cenários de testados. Ressalta-se que o vídeo de demonstração anexado ao trabalho contém uma quantidade ainda maior de testes executados, evidenciando a robustez e o correto funcionamento do sistema.

6.1 Teste 1 – Vitória na Partida



Figura 2: Partida finalizada com vitória

Neste cenário, o jogador conseguiu descobrir corretamente a palavra secreta dentro do limite de seis tentativas. O sistema identificou a vitória, exibiu corretamente as cores

correspondentes às letras e encerrou a partida.

Além disso, imediatamente após a vitória, as estatísticas do jogador foram atualizadas, incrementando o número total de partidas jogadas e o número de vitórias.

6.2 Teste 2 – Acesso às Estatísticas



Figura 3: Tela de estatísticas após vitória

Após vencer a partida, o jogador acessou a tela de estatísticas. Foi possível verificar que os dados foram corretamente atualizados, refletindo o resultado obtido na partida

recém-finalizada.

6.3 Teste 3 – Persistência Após Reinicialização

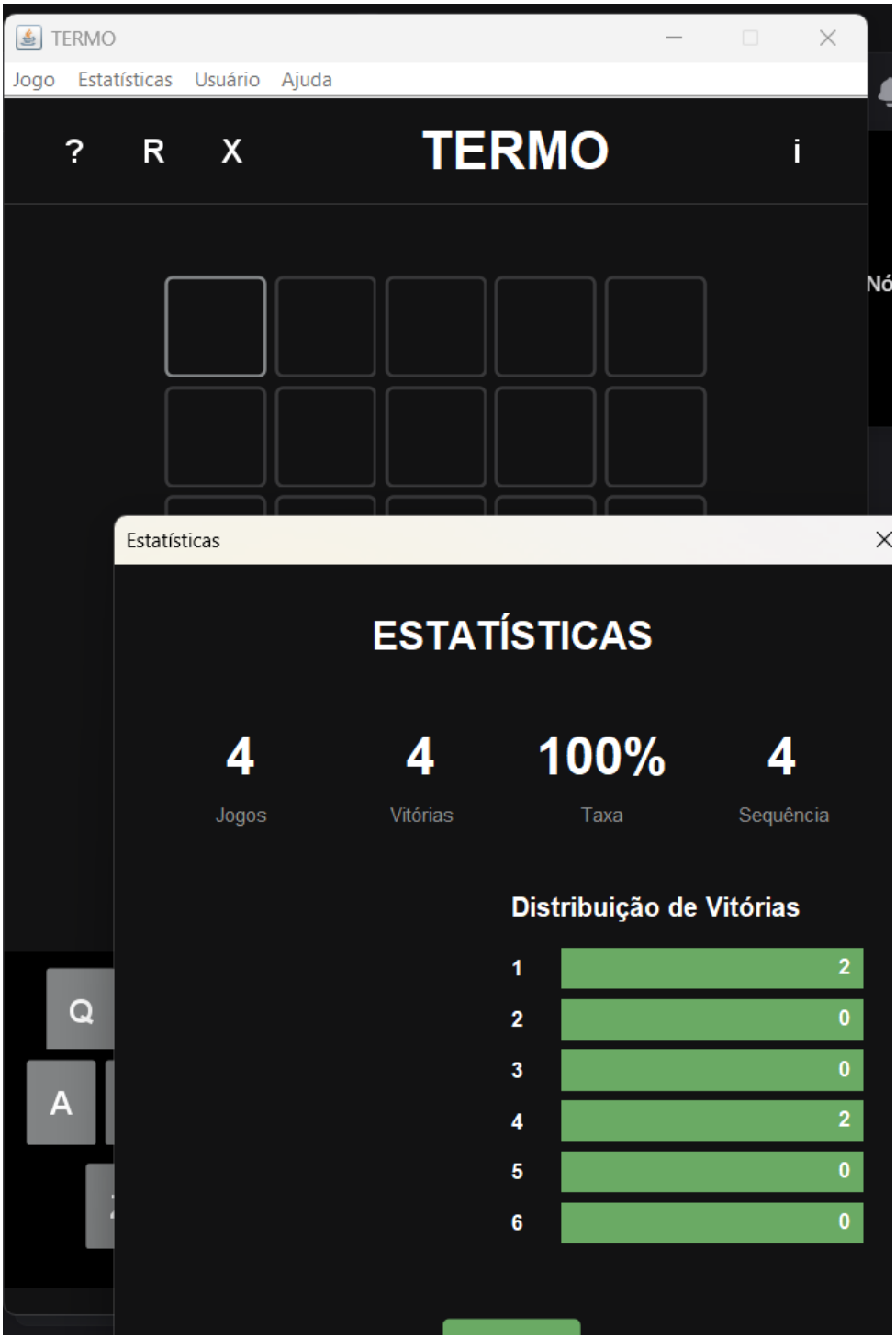


Figura 4: Estatísticas mantidas após reinicialização do sistema

Neste teste, o sistema foi completamente encerrado e posteriormente reiniciado. Após realizar login novamente, as estatísticas permaneceram inalteradas, comprovando o correto funcionamento da serialização e da persistência em arquivo binário.

Esse teste confirma que os dados não são mantidos apenas em memória, mas efetivamente gravados e recuperados do arquivo.

6.4 Teste 4 – Login com Senha Incorreta



Figura 5: Tentativa de login com senha incorreta

Foi realizada tentativa de autenticação utilizando senha incorreta. O sistema bloqueou o acesso e exibiu mensagem apropriada ao usuário.

Esse teste valida o funcionamento do mecanismo de autenticação e confirma que as senhas são comparadas por meio de hash, sem armazenamento em texto plano.

6.5 Teste 5 – Palavra Fora do Dicionário

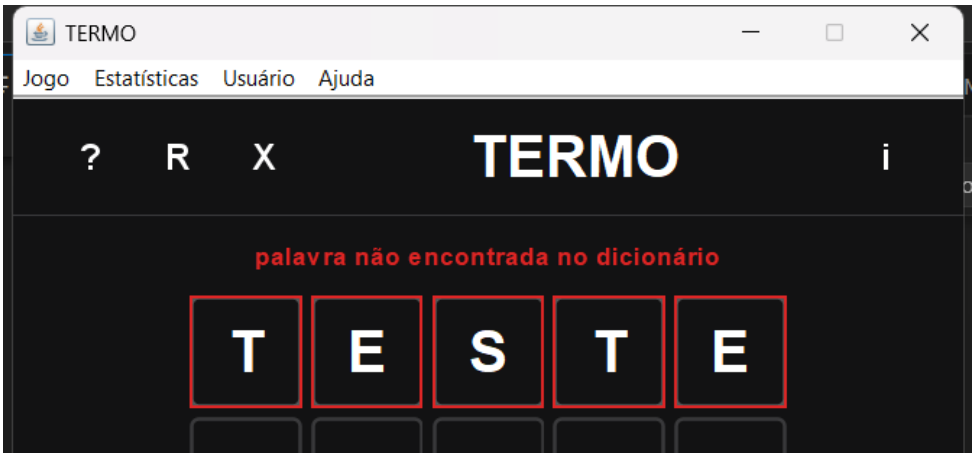


Figura 6: Tentativa de inserir palavra inexistente no banco

Neste cenário, foi digitada uma palavra inexistente no banco de palavras. O sistema exibiu mensagem de erro informando que a palavra não foi encontrada e não consumiu tentativa.

Esse comportamento confirma que a validação do banco de palavras está sendo realizada corretamente antes do processamento da tentativa.

7. CONCLUSÃO

O desenvolvimento do jogo TERMO permitiu aplicar de forma prática e integrada os principais conceitos de Programação Orientada a Objetos estudados na disciplina.

Entre os conhecimentos consolidados destacam-se:

- Encapsulamento e organização de responsabilidades.
- Modelagem com composição entre objetos.
- Separação arquitetural por meio do padrão MVC.
- Implementação de persistência utilizando serialização.
- Uso de hash SHA-256 para armazenamento seguro de senhas.

As principais dificuldades encontradas durante o desenvolvimento envolveram:

- Implementação correta da lógica de cores considerando letras repetidas.
- Customização avançada da interface Swing, incluindo ajuste de cores e renderização personalizada.
- Garantia de atualização correta do teclado virtual.
- Controle adequado de foco e eventos da interface gráfica.
- Implementação consistente da persistência de dados.

Ajustes finos foram necessários para que as regras de cores seguissem corretamente a lógica do jogo original, evitando que letras repetidas fossem marcadas incorretamente.

Como melhorias futuras, destacam-se:

- Implementação de testes automatizados (JUnit).
- Integração com banco de dados relacional.
- Interface gráfica mais moderna com animações.
- Sistema de ranking entre usuários.

O resultado final é um sistema completo, funcional e organizado, atendendo plenamente aos requisitos propostos e demonstrando domínio dos conceitos de Programação Orientada a Objetos.